University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Assignment 1 - Report

Gianluca Panzani

April 9, 2026

Contents

1 Implementation **1**
1.1 Execution variants and workflow . . . 1
1.2 Experimental grid and reproducibility . . . 1
1.3 Choice of the number of partitions `P` . . . 1
1.4 Hash functions . . . 1
1.5 Correctness verification . . . 2

2 SIMD Optimization **2**
2.1 Vectorization evidence and hash selection . . . 2
2.2 AVX2 mask kernel . . . 2

3 Results Analysis **2**
3.1 Benchmarking methodology . . . 3
3.2 Performance and limits . . . 3
3.3 Speedup . . . 4

4 Conclusions **4**

1 Implementation

1.1 Execution variants and workflow

The project benchmarks the partition phase of a Partitioned Hash Join with three executables: `plain_novec`, `plain_vec`, and `avx2`. The three builds share the same algorithmic structure and interface but differ in vectorization strategy.

Each run receives `N`, `P`, `hash_name` and the path `output_csv`; then it loads datasets, computes partitions for both relations, measures partition/global time, computes checksums and appends one result row to CSV.

This setup was chosen to keep the comparison controlled: all variants execute the same workflow on the same node (`node09`), consume the same input files and produce the same output format. Therefore, timing differences are mainly attributable to code generation and kernel implementation choices, not to different data paths or instrumentation overhead.

1.2 Experimental grid and reproducibility

The automation uses Bash + Slurm scripts. The file `run_grid_search.sh` handles cleanup, compilation, dataset generation and dependent submissions for `plain_novec` → `plain_vec` → `avx2`.

Main grid (`runners/grid_config.sh`):

- `N` $\in \{10^5, 10^6, 10^7\}$,
- `P` $\in \{16, 32, \ldots, 4096\}$,
- hash $\in \{$`mask`, `xorshift`, `fmix32fold`$\}$.

This gives 81 configurations per executable. To evaluate the consistency of the measurements, we fixed `N`=10^7 and repeated each configuration `10` times.

AVX2 currently supports only `mask` hash function, for the other cases the AVX2 executable exits after the runtime check.

1.3 Choice of the number of partitions `P`

`P` is restricted to powers of two (checked at runtime). This enables partition-id computation with bit masking:

$$\texttt{part_id} = \texttt{hash} \ \& \ (\texttt{P} - 1),$$

which avoids modulo/division in the hot loop and is SIMD-friendly.

The explored range `16--4096` (4–12 partition bits) is wide enough to test coarse vs fine partitioning while keeping the grid manageable. Results show that within this range the impact of `P` is limited (max/min mean partition-time ratio about 1.001–1.008), while hash complexity has a much larger effect.

1.4 Hash functions

Initial 64-bit-multiply-based mappings were also auto-vectorized by GCC, but were not performance-profitable end-to-end. The final selected hashes rely on simpler bitwise and 32-bit-friendly operations, better suited to efficient auto-vectorization on the target CPU.

So at the end, the following three hashes are benchmarked:

- `mask`: low-bit selection via AND with `P-1` (lowest arithmetic cost),
- `xorshift`: 64-bit shift/XOR mixing before masking,

- `fmix32fold`: 64-to-32 fold plus fmix32-style finalization with shifts/XORs and 32-bit multiplications.

This choice spans a useful tradeoff space: `mask` maximizes mapping speed, while `xorshift` and `fmix32fold` provide progressively stronger bit mixing and typically more robust partition distribution on non-ideal key patterns.

1.5 Correctness verification

Correctness is validated through checksum comparison: each run stores a checksum of partitioned outputs, enabling cross-implementation validation without printing arrays.

All optimized variants were checked and, in each generated `.csv` file, every comparable group has a single checksum.

This confirms that all implementations produce the same partitioning result for the same input, which gives an enough reliable indication of correctness.

2 SIMD Optimization

2.1 Vectorization evidence and hash selection

Vectorization is inspected with GCC diagnostics. The executables `plain_vec` and `avx2` are got by compiling with `-fopt-info-vec-optimized -fopt-info-vec-missed`; while the `plain_novec` one uses `-fno-tree-vectorize`.

Representative lines from `gcc_vec_report.txt` confirm the auto-vectorization of the main partitioning loops in `plain_vec`:

```
lib/partition.cpp:9:31: optimized: loop vectorized using 32 byte vectors
lib/partition.cpp:20:31: optimized: loop vectorized using 32 byte vectors
lib/partition.cpp:36:31: optimized: loop vectorized using 32 byte vectors
```

2.2 AVX2 mask kernel

The AVX2 kernel in `lib/partition_avx2.cpp` processes 4 keys per iteration with:

1. vector load of 4 x 64-bit keys,
2. vector AND with broadcast `P-1`,
3. packing low 32-bit lanes,
4. 128-bit store of 4 partition ids,
5. scalar tail handling.

3 Results Analysis

All quantitative values are computed from `results/benchmark_results.csv` (repeated runs), while `results/results.csv` is used only for the `N`-scaling observation.

3.1 Benchmarking methodology

The repeated benchmark uses `M=10` repetitions per configuration. We report execution-time means and standard deviations for the main comparisons, and we also report medians for the mask summary to satisfy robust-timing requirements.

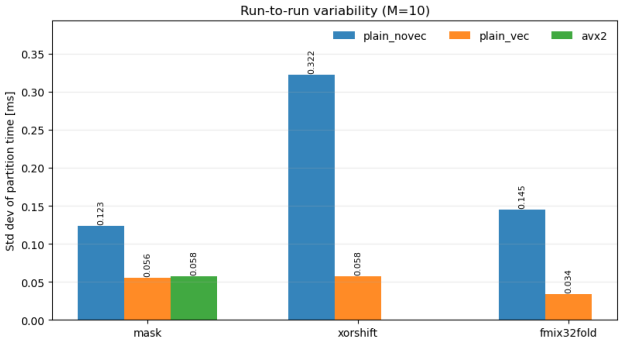

Figure 1: Standard deviation of partition time across the 10 repetitions.

The variability values are small relative to mean partition times. For the `mask` summary, standard deviations are around 0.123 ms (`plain_novec`), 0.056 ms (`plain_vec`), and 0.058 ms (`avx2`), corresponding to sub-1% relative variation. This supports using mean values as representative central estimates.

3.2 Performance and limits

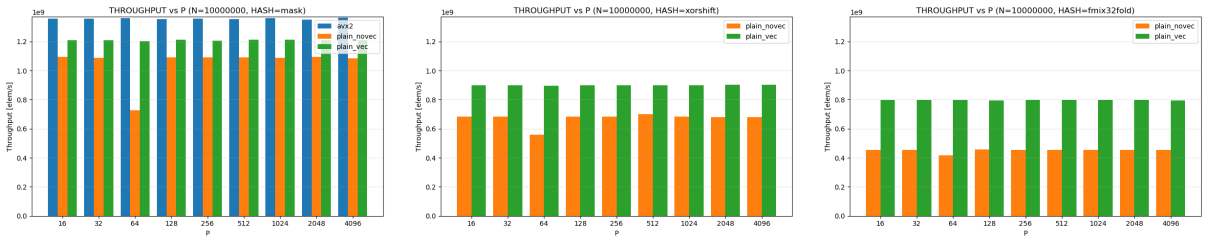

Figure 2: Throughput vs `P` for `mask`, `xorshift`, and `fmix32fold`.

Average throughput ranking is stable: `mask` > `xorshift` > `fmix32fold`.

- `plain_novec`: $1.09 \cdot 10^9$ (`mask`), $6.86 \cdot 10^8$ (`xorshift`), $4.55 \cdot 10^8$ (`fmix32fold`) elems/s,
- `plain_vec`: $1.21 \cdot 10^9$ (`mask`), $8.98 \cdot 10^8$ (`xorshift`), $7.96 \cdot 10^8$ (`fmix32fold`) elems/s,
- `avx2`: $1.36 \cdot 10^9$ (`mask`) elems/s.

Limits interpretation from observed speedups:

- `mask` has low arithmetic intensity and is closer to memory-bound behavior (modest SIMD gain),
- `xorshift` is more compute-heavy and gets larger SIMD gain,
- `fmix32fold` is the most compute-heavy and achieves the highest relative SIMD gain.

For `mask`, `results/results.csv` also shows overhead amortization with `N`: throughput rises from about 9.95×10^8 at `N=`10^5 to 1.09×10^9 elems/s at `N=`10^7 in `plain_novec`.

3.3 Speedup

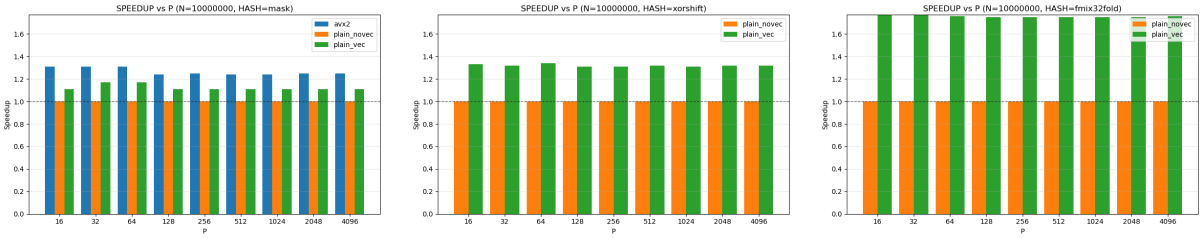

Figure 3: Speedup vs `P` (baseline: `plain_novec`).

AVX2 is unsupported for `xorshift` and `fmix32fold`; in those cases speedup comes from `plain_vec` only. Mean speedups of `plain_vec` over `plain_novec` are about 1.11× (`mask`), 1.31× (`xorshift`), and 1.75× (`fmix32fold`).

`exec_type`	`partition_time` [s] (mean $\pm\sigma$)	`throughput` [elems/s]	`speedup`
`plain_novec`	0.018373 ± 0.000123	1.089×10^9	1.0000
`plain_vec`	0.016536 ± 0.000056	1.210×10^9	1.1111
`avx2`	0.014721 ± 0.000058	1.359×10^9	1.2481
All executions in this batch were run on the same machine (`node09`).			

Table 1: `Mask`-only repeated benchmark summary.

4 Conclusions

Reading throughput and speedup together is important: absolute throughput is highest for `mask`, but relative SIMD gain is highest for `fmix32fold`. This is coherent with a classic roofline-style intuition. When per-element arithmetic is very small (`mask`), memory traffic quickly becomes the dominant cost and extra SIMD compute capacity cannot be fully exploited. As arithmetic work increases (`xorshift`, then `fmix32fold`), vector lanes are used more effectively and the benefit of SIMD instructions becomes more visible in the speedup metric.

At the same time, the near-flat curves across `P` indicate that, in the explored range, changing the number of partitions does not materially change the computational character of the kernel.

So, the dominant factors remain hash complexity and execution strategy (scalar, auto-vectorized, or manual AVX2), and the results are consistent across runs and configurations:

- hash complexity dominates absolute cost (`mask` fastest, `fmix32fold` slowest),
- SIMD gains depend on kernel type: smaller on memory-lean `mask`, larger on compute-heavier hashes,
- within `P=16..4096`, the effect of `P` is secondary compared to hash and execution kernel.

For the AVX2 manual path, we have a higher speedup with respect to the auto-vectorized version. This could be due to the fact that the hand-written AVX2 kernel removes part of the compiler-generated overhead and uses a SIMD sequence that is more directly matched to the structure of the mask operation.